TECHNICAL REPORT

Coaching & Theoretical Engine Improvement Report

Cross-referencing 257 FSAE vehicle dynamics resources against the Lap Coach telemetry analysis platform to identify high-impact upgrades

Platform: Lap Coach Racing — Motorsport Telemetry SaaS

Source Material: 81 MATLAB scripts, 51 PDFs, 10 Excel tools (FSAE library)

Codebase Analyzed: kart_envelope/ coaching pipeline, telemetry_tool/ solver & scoring

Date: 13 March 2026

Classification: Internal — Engineering

Table of Contents

Executive Summary

Part 1: Tire Model Upgrades 1a–1c

Part 2: Vehicle Dynamics Model Upgrades 2a–2d

Part 3: Racing Line & Curvature Improvements 3a–3c

Part 4: Coaching Engine Intelligence 4a–4e

Part 5: Theoretical Engine Accuracy 5a–5e

Part 6: New Capabilities 6a–6e

Part 7: Priority Ranking

Part 8: Architecture Recommendation

0 Executive Summary

This report cross-references a comprehensive FSAE vehicle dynamics library (257 files spanning tire modeling, ISO testing standards, lap time simulation, and suspension analysis) against the current Lap Coach coaching and theoretical engine codebase to identify every actionable improvement opportunity.

Key finding: The current engine optimizes *speed on a fixed path* but never optimizes *the path itself*. Racing line accounts for 60–70% of the lap time difference between amateur and professional drivers. This is the single highest-value gap.

23 improvements identified, organized into 5 priority tiers:

PRIORITY	COUNT	THEME	EXPECTED IMPACT
P0	2	Racing line optimization, cross-session progression	Transformative — addresses primary skill gap
P1	4	Multi-apex coaching, brake modulation, tire physics	High — 3–8% accuracy, new coaching dimensions
P2	5	Adaptive thresholds, G-G analysis, tire degradation	Medium — eliminates false positives, endurance support
P3	6	Elevation, reference drivers, steering analysis, roll	Moderate — circuit-specific and advanced coaching
P4	6	Frequency response, setup sensitivity, predictive	Low/Niche — requires full vehicle model

The improvements cluster into **three parallel implementation streams**: Physics Fidelity (tire models + OCP solver), Coaching Intelligence (pattern detection + progression), and Racing Line (path optimization as preprocessing).

1 Tire Model Upgrades

Current State

> **Quantile-based capability envelope** — 99th percentile of observed ax/ay per 5 km/h speed bin, with a p-norm friction ellipse (p≈1.6). This is data-driven but has no physics basis for tire behavior. The envelope conflates vehicle capability with driver capability.

FSAE Reference Material

MODEL	SOURCE FILE	COMPLEXITY	WHAT IT ADDS
Pacejka 4-term	`pacejka4.m`	Low	FY(α) = D·sin(C·atan(B·α)), cornering stiffness at small slip
Pacejka 5-term	`Pacejka5/`	Medium	Additional Fy2 term for extreme slip angles
MF-Tyre 5.2	`MF52_FY.m` , `MF52_MX.m` , `MF52_MZ.m`	High	Combined cornering+braking, temperature, pressure, load variation

1a. Replace fixed friction circle with load-sensitive Pacejka model

> **Current:** `ay_max` is a flat number per speed bin.
> **Better:** `ay_max(Fz) = D1·Fz + D2·Fz²` where Fz changes with load transfer.
> **Source:** FSAE `nonlin.m` shows exactly how cornering compliance DF/DR change with lateral acceleration due to load transfer — the current engine ignores this entirely.

Impact: 3–8% improvement in theoretical lap accuracy on high-downforce cars; matters less for karts.

1b. Add combined-slip tire model

> **Current:** p-norm ellipse `(ax/ax_max)^p + (ay/ay_max)^p ≤ 1` with fixed p=1.6.
> **Better:** MF-Tyre 5.2 combined slip model (`wsu.m`) accounts for tire saturation differently under braking-while-turning vs acceleration-while-turning.
> **Key finding:** Lateral force FY drops *faster* under braking than under traction — the current symmetric p-norm misses this asymmetry.

Impact: More accurate trail-braking coaching. Currently the engine can't distinguish "at the grip limit" from "15% more lateral grip available because you're off the brakes."

1c. Add tire transient lag model

> **Source:** PDF "Tire Transient Lateral Force Generation" documents that tires take 0.05–0.15s to build lateral force after slip angle change (relaxation length effect).
> **Current:** Assumes instantaneous grip — theoretical speeds through quick direction changes (chicanes, slaloms) are too optimistic.

Impact: 50–150ms more realistic theoretical time through chicanes; better coaching for quick transitions.

> **★ ENGINEERING INSIGHT**
>
> The fundamental limitation isn't that you lack tire data — it's that the quantile envelope conflates vehicle limits with driver limits. A driver who never trail-brakes produces an envelope that *appears* to have lower combined-slip grip than the vehicle actually has. Pacejka separates vehicle capability from driver behavior.

2 Vehicle Dynamics Model Upgrades

Current OCP Solver Models

> **Implemented in `ocp_solver.py` :** Energy balance (v^2 dynamics), friction ellipse constraint, basic front/rear Fz load transfer, aerodynamic drag + downforce (constant Cd/Cl), throttle-brake complementarity.

2a. Add understeer gradient K to theoretical model

Source: `nonlin.m` and `marc1-5.m` compute K (deg/g) from ISO 4138 data.

> **Current:** Model assumes the vehicle can achieve any lateral g at any speed if grip allows.
> **Better:** Add steering constraint: `δ_required = (L/R) + K·ay`. If `δ_required > δ_max` (steering lock), corner speed is limited by steering, not grip.

Impact: Critical for cars with high understeer (most production cars, some FSAE cars). Less important for karts.

2b. Add roll dynamics

Source: `roll_play.m` — roll gain `rg = m_sprung · h_sprung / k_total` and roll stiffness distribution.

```
Roll angle: ϕ = ay · rg Load transfer: ΔFz = m·ay·h/t + m·ay·ϕ·d_roll
```

Impact: 1–3% theoretical accuracy improvement for cars with soft suspension; negligible for karts.

2c. Add frequency response characterization

Source: `chirp_test.m` and `joao_proc.m` extract transfer functions (yaw rate/steer, sideslip/steer) and transient metrics (rise time, damping ratio, overshoot).

> **New coaching dimension:** Compare driver's steering input frequency spectrum to the vehicle's natural frequency. If dominant input frequency is near yaw natural frequency (1–3 Hz), vehicle amplifies oscillations — flag "your steering corrections are too frequent."

Impact: Opens a completely new coaching category: driver-vehicle frequency mismatch.

2d. Add Ackermann steering geometry effects

Source: `ackermannError.xlsx` — difference between ideal Ackermann angle and actual outer wheel angle.

Impact: Small (1–2%) but noticeable on tight kart circuits with many hairpins.

★ ENGINEERING INSIGHT

The FSAE toolkit follows a systematic validation workflow: ISO 4138 test → extract K → validate against chirp/step response → feed into lap sim. Your engine skips the first three steps. Adding K extraction from the driver's own telemetry (curvature + ay + steering angle) would create a vehicle characterization step that makes the theoretical lap physically grounded rather than purely statistical.

3 Racing Line & Curvature Improvements

Current State

> **Track geometry derived from driver's GPS path.** Curvature `κ = d(heading)/ds` computed from driver data, so the "theoretical line" is actually "theoretical speed on the driver's actual line."

3a. Separate racing line optimization from speed optimization P0

Source: `curvature.m` (circumcenter method) + Track_Attack + MaxLat_Sim.

> **This is the single highest-value improvement.** Racing line accounts for 60–70% of lap time delta between amateur and pro drivers; speed execution on a given line is only 30–40%.
>
> **Two-stage optimization:**
> (1) Find optimal path through track boundaries (geometric optimization)
> (2) Find optimal speed on that path (dynamic optimization — current system)
> Currently only step 2 exists.

Implementation: Given track boundaries (derived from GPS cloud of all laps), compute the optimal racing line that minimizes curvature through corners while maximizing straight length.

3b. Multi-apex corner handling P1

> **Current:** Curvature threshold + merge heuristic (<50m apart). Chicanes appear as 2–3 separate corners.
> **Better:** Classify corner sequences as `single`, `chicane`, `esses`, `decreasing_radius`, `increasing_radius`.
> **New coaching:** "Sacrifice corner 3 entry to maximize corner 4 exit speed."

Impact: Addresses one of the biggest coaching blind spots. Pro drivers think in corner complexes, not individual corners.

3c. Elevation-aware curvature

> **Current:** 2D GPS curvature. Hills create false curvature in heading signal.
> **Better:** `curvature.m` supports 3D path points — separate lateral curvature from vertical. Use vertical curvature for crest/compression grip effects.

Impact: Critical for hillclimb tracks and circuits with elevation (Spa, Bathurst, Road America).

> **★ ENGINEERING INSIGHT**
> The FSAE `curvature.m` uses the circumcenter method (find circle through 3 consecutive points). This is more robust than the current `dheading/ds` approach because it's less sensitive to GPS noise — heading derivatives amplify noise, while circumcenter averages it across 3 points.

4 Coaching Engine Intelligence

Current Limitations

LIMITATION	CURRENT BEHAVIOR	CONSEQUENCE
Fixed thresholds	`brake_util_p90 < 0.88` = underbraking (same for F2000 and 125cc kart)	False positives/negatives across vehicle types
No cross-session learning	Each session analyzed in isolation	Can't track improvement over sessions
Zone averaging	Entry/apex/exit speeds averaged across zone	Can't distinguish smooth vs abrupt braking
20 fixed segments	Same granularity for 400m and 6km tracks	Over-granular or under-granular
No line analysis	Only speed delta, not position delta	Misses primary skill difference
Single-lap diagnostics	Diagnostic runs on one lap	Misses session-level patterns

4a. Adaptive thresholds by vehicle class P2

Source: `BILLCOBB_VD101.xlsm` contains Pacejka coefficients for 50+ tire sets and parameters for 20+ vehicles. Lateral g ranges from 0.6g (road car) to 2.5g (FSAE with downforce).

> **Create vehicle class presets:** `kart_125cc`, `kart_rotax`, `car_touring`, `car_formula`, `car_gt` with scaled thresholds.
> `braking_threshold = base_threshold × vehicle_class.brake_factor`

4b. Steering input analysis (new coaching dimension)

Source: `joao_proc.m` extracts response times (tay, tbeta, tyawv) and damping ratio.

> **If RaceChrono records steering angle, analyze:**
>
> - Steering smoothness (frequency content of steering signal)
> - Correction frequency (how many mid-corner corrections per lap)
> - Steering lead vs lag (does driver anticipate or react?)
>
> **New pattern:** `reactive_steering` vs `anticipatory_steering`

Impact: Steering technique is the #1 differentiator between intermediate and advanced drivers.

4c. Brake modulation analysis P1

Source: FSAE frequency response tests show how vehicle response changes with input rate.

New patterns:

- `abrupt_braking` — Pressure ramp <0.1s from 0 to peak (lockup risk)
- `trailing_brake_absent` — No gradual release into corner entry (missing trail-brake)
- `late_peak_brake` — Peak decel >20% into braking zone

Currently: Engine only knows "braked too early" or "braked too late" — not *how* the driver braked.

4d. Speed-dependent G-G utilization P2

Source: `FSAE_MaxLat_Sim.m` maps maximum ay at each speed.

Current: Compares driver's G-G scatter to a single envelope circle.
Better: Speed-binned G-G analysis reveals:
• Low-speed corners: driver uses 90% of grip (good)
• High-speed corners: driver uses only 60% (trust issue)
New pattern: `speed_dependent_trust_deficit`

Impact: Reveals psychological limits vs physical limits.

4e. Cross-session progression tracking P0

Store per-corner and per-pattern metrics with session timestamps. Compute:

- Corner-specific improvement trend ("Turn 3 time loss: 320ms → 180ms over 4 sessions")
- Pattern resolution ("early_braking pattern resolved after session 3")
- Skill profile evolution (radar chart across sessions)

Impact: Transforms coaching from "what you did wrong today" to "how you're developing as a driver."

★ ENGINEERING INSIGHT

The FSAE workflow follows a test-validate-simulate cycle. Your engine currently only does "simulate" (theoretical) and "report" (coaching text). Adding "validate" (compare the vehicle model against real session data to check model accuracy before using it for coaching) would dramatically improve coaching credibility. A theoretical lap that's 3% wrong produces actively misleading advice.

5 Theoretical Engine Accuracy

5a. Better warm-start from curvature

> **Current:** `v_init = sqrt(μ · g / κ)` — pure lateral limit, ignores aero downforce at speed.
> **Better:** `v_init[i] = min(v_qss[i], sqrt((μ·g + 0.5·ρ·Cl·A·v²/m) / κ[i]))`

5b. Variable friction coefficient

> **Current:** Single `μ` for entire track.
> **Better:** Build distance-dependent `μ(s)` map from multi-session corpus. Grip at corner entry (clean asphalt) differs from apex (rubber-ed in) differs from kerbs (painted, low grip).

Impact: 2–5% more accurate theoretical through corners where grip varies significantly.

5c. Wind model

```
F_aero = 0.5 · ρ · Cd · A · (v + v_wind)²
```

Headwind/tailwind affects lap times by 0.5–2%. If weather data is available, adjust theoretical accordingly.

Impact: Explains session-to-session variation that currently appears as "driver inconsistency."

5d. Tire degradation model P2

> **Current:** Single envelope snapshot per session.
> **Better:** Track envelope evolution across laps: train separate envelopes for lap 1–5, 6–15, 16–30, 30+.
> **New coaching:** "You're pushing too hard in the first 5 laps — you'll have 8% less grip by lap 20."

Impact: Critical for endurance sessions (20+ laps).

5e. Solver convergence improvements

> **Current:** CasADi/IPOPT falls back to QSS on failure (~15% of laps).
> **Better:**
> - Multiple constraint relaxation levels: try tight first, relax progressively
> - Proper IPOPT warm-start: `warm_start_init_point: yes` with lambda/multiplier guesses from QSS
> - Set `mu_init = 0.1` for faster initial progress

Impact: Reduces fallback-to-QSS rate from ~15% to ~3%.

6 New Capabilities (Not in Current Engine)

6a. ISO 4138 Vehicle Characterization from Telemetry

Extract vehicle handling characteristics from the driver's own telemetry data:

- From constant-speed cornering data, compute understeer gradient K
- From rapid direction changes, estimate yaw natural frequency and damping
- From braking events, estimate brake balance front/rear

Output example: "Your vehicle has K=2.1 deg/g (moderate understeer), yaw natural frequency 1.8 Hz, 58% front brake bias."

Value: Gives mechanics actionable setup feedback alongside driver coaching.

6b. Setup Change Sensitivity Analysis

Run theoretical lap with varied vehicle parameters: +10% front spring rate, -5% rear anti-roll bar, etc. Show predicted lap time delta for each change.

Requires: Vehicle model with Pacejka tire parameters (not just empirical envelope).

Value: Bridges gap between "what the driver should do" and "what the car setup should do."

6c. Sector-Level Strategy Coaching P2

Instead of corner-by-corner (tactical), provide sector-level strategy:

- "Sector 1 (slow corners): you lose 320ms. This is a minimum-speed technique issue."
- "Sector 2 (high-speed): you lose only 40ms. Your confidence at speed is strong."
- "Sector 3 (mixed): you lose 200ms, mostly on exits. Get to full throttle earlier."

Value: Helps drivers prioritize where to focus practice time.

6d. Reference Driver Comparison

Store "gold standard" laps from fast drivers on the same track. Compare user telemetry against a reference driver — not just the theoretical.

Value: Theoretical = "perfect robot"; reference driver = "achievable human." Much more motivating.

6e. Predictive Lap Time Calculator

Given a new track layout (GPS trace), predict the user's likely lap time based on their performance envelope from previous tracks.

Example: "Based on your driving profile, expect 1:23.5 at Spa vs your current 1:26.2 — high-speed cornering confidence is costing ~2.7s."

7 Priority Ranking

Ranked by **(impact on coaching quality) × (feasibility with current architecture)**:

PRIORITY	IMPROVEMENT	IMPACT	EFFORT	KEY FILES
P0	**Racing line optimization** (3a)	Massive — 60–70% of lap time gap	HIGH	New: `line_optimizer.py`
P0	**Cross-session progression** (4e)	Transforms product value	MED	DB schema + new `progression.py`
P1	**Multi-apex corner coaching** (3b)	Fixes biggest coaching blind spot	MED	`corner_coaching.py`
P1	**Brake modulation analysis** (4c)	Most actionable coaching	MED	`driver_diagnostics.py`
P1	**Load-sensitive tire model** (1a)	3–8% theoretical accuracy	MED	`capability.py`, `ocp_solver.py`
P1	**Combined-slip asymmetry** (1b)	Better trail-braking coaching	MED	`capability.py`
P2	Adaptive thresholds by class (4a)	Eliminates false positives	LOW	`driver_diagnostics.py`
P2	Speed-dependent G-G (4d)	Reveals trust issues	LOW	`driver_diagnostics.py`
P2	Tire degradation tracking (5d)	Critical for endurance	MED	`capability.py`, new `degradation.py`
P2	Understeer gradient K (2a)	Physical grounding for cars	MED	`ocp_solver.py`, new `vehicle_char.py`
P2	Sector strategy coaching (6c)	Better driver prioritization	LOW	`cause_ranking.py`
P3	Elevation-aware curvature (3c)	Important for hilly circuits	LOW	`track_geometry.py`
P3	Reference driver comparison (6d)	Motivational coaching	MED	DB + new `reference.py`
P3	Steering input analysis (4b)	New coaching dimension	MED	`driver_diagnostics.py`
P3	Roll dynamics (2b)	Small accuracy gain	MED	`ocp_solver.py`
P3	Variable friction map (5b)	Track-specific accuracy	MED	`capability.py`
P3	Wind model (5c)	Explains variation	LOW	`ocp_solver.py`
P4	Frequency response coaching (2c)	Advanced coaching	HIGH	New module

P4	Vehicle characterization (6a)	Setup tool	HIGH	New module
P4	Setup sensitivity (6b)	Requires full vehicle model	V. HIGH	New module
P4	Tire transient lag (1c)	Small accuracy gain	MED	`capability.py`
P4	Predictive lap time (6e)	Nice-to-have	HIGH	New module
P4	Ackermann correction (2d)	Marginal	LOW	`ocp_solver.py`

8 Architecture Recommendation

The 23 improvements cluster into three streams that can proceed in parallel:

Stream A: Physics Fidelity

Scope: Items 1a, 1b, 2a → `capability.py` + `ocp_solver.py`

Add Pacejka-based tire model as an *alternative* to the quantile envelope. Keep quantile as fallback when no tire data is available. Wire Pacejka through the same `CapabilityEnvelope` interface so the solver doesn't need to know which model is active.

Key principle: Two-tier fidelity — empirical envelope (always available) vs physics model (when vehicle parameters are known).

Stream B: Coaching Intelligence

Scope: Items 3b, 4a–4e → `coaching/` package

Extend the existing coaching pipeline with new pattern detectors and cross-session storage. No architectural changes needed — add new methods to `driver_diagnostics.py` and new DB tables for progression tracking.

Key principle: Each new pattern detector is additive and non-breaking. Deploy incrementally.

Stream C: Racing Line Optimization

Scope: Item 3a → new `line_optimizer.py`

The largest change. Requires: track boundary extraction from GPS cloud → convex optimization for minimum-curvature path → integration with existing speed optimizer. Implemented as a **preprocessing step** before the theoretical solver runs.

Key principle: Racing line is computed once per track (not per session), cached, and reused. The speed optimizer takes the optimal line as input instead of the driver's GPS line.

FSAE Source Material Summary

CATEGORY	FILES	KEY RESOURCES
Tire Models	Pacejka4/, Pacejka5/, MF5.2/	`pacejka4.m`, `MF52_FY.m`, `wsu.m`
Vehicle Dynamics	Matlab Sandbox/, nonlin_tr/	`nonlin.m`, `chirp_test.m`, `roll_play.m`
Racing Line	Curvature/, Track_Attack/	`curvature.m`, `FSAE_MaxLat_Sim.m`
Response Analysis	JOAO/	`joao_proc.m`
Test Data	Root + Matlab Sandbox	`BILLCOBB_VD101.xlsm`, 4x test proc .xlsx
Theory & Standards	Publications/, Testing/	51 PDFs: ISO 4138–9816, MF-Tyre 5.2 manual, F1 insights